

CSA 0613-DESIGN AND ANALYSIS OF ALGORITHMS

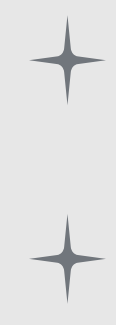
Comparing The Best, Average, And Worst Case Analysis Of The Quick Sort Algorithm Using Asymptotic Notation



Presented By:
KALPANA J(192521038)



INTRODUCTION

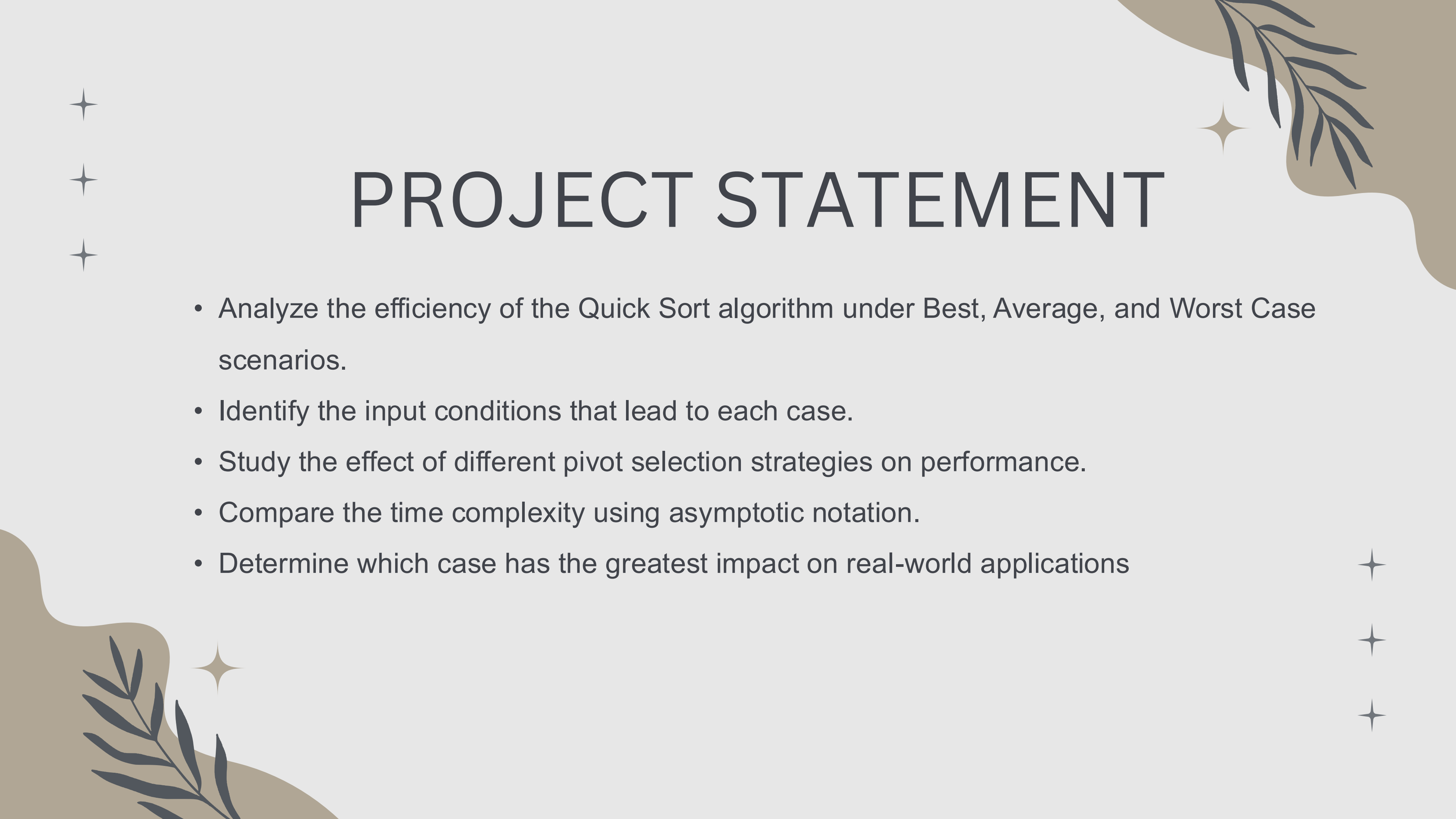
- 
- Sorting is the process of arranging data in a specific order, such as ascending or descending, to improve data organization and searching efficiency.
 - Quick Sort is one of the most efficient and widely used sorting algorithms based on the Divide and Conquer strategy.
 - The performance of Quick Sort depends significantly on the choice of the pivot element
 - The three important cases considered are:

Best Case – Balanced partitioning of elements.

Average Case – Randomly distributed elements resulting in reasonably balanced partitions.

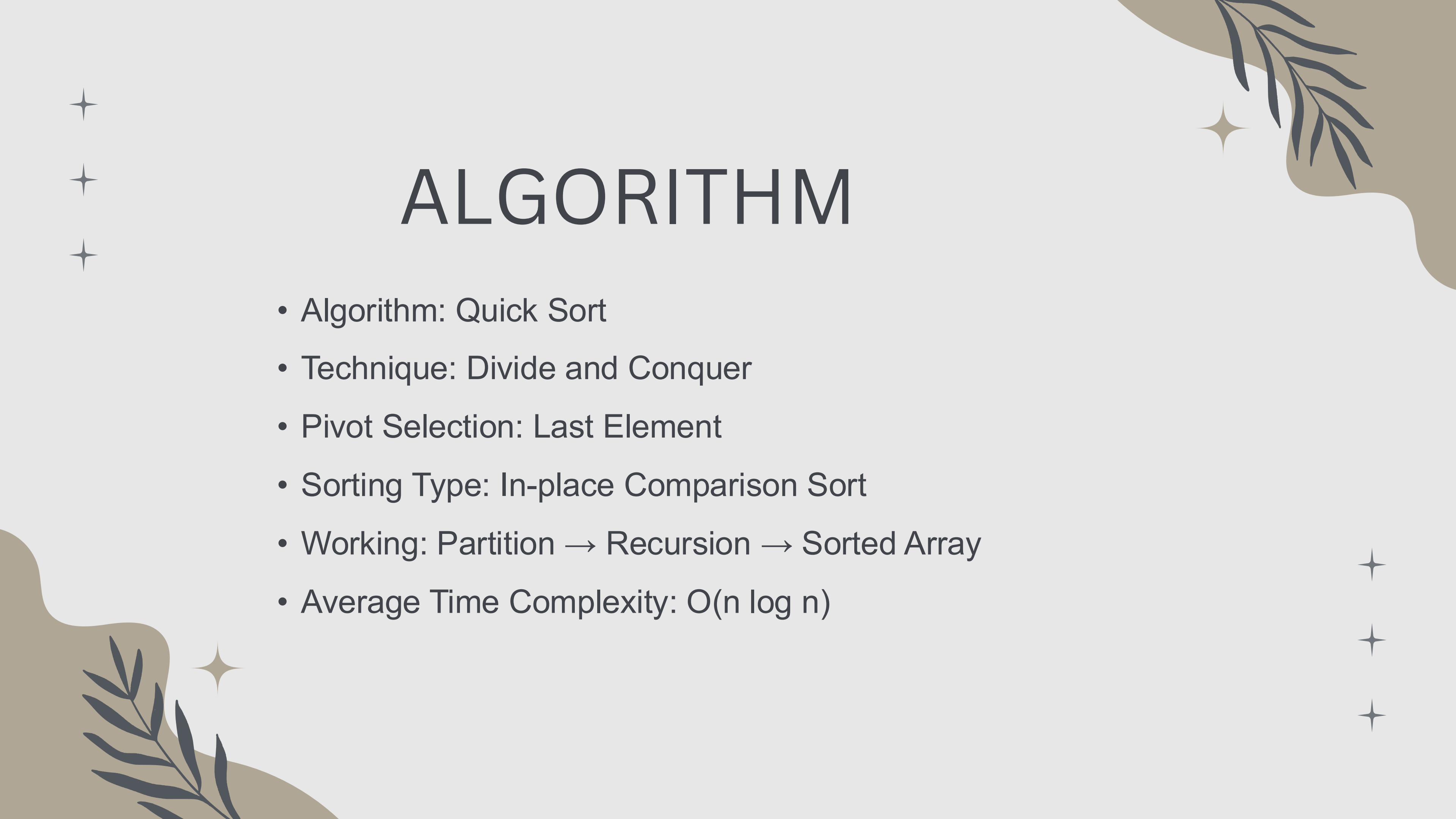
Worst Case – Highly unbalanced partitions caused by poor pivot selection.





PROJECT STATEMENT

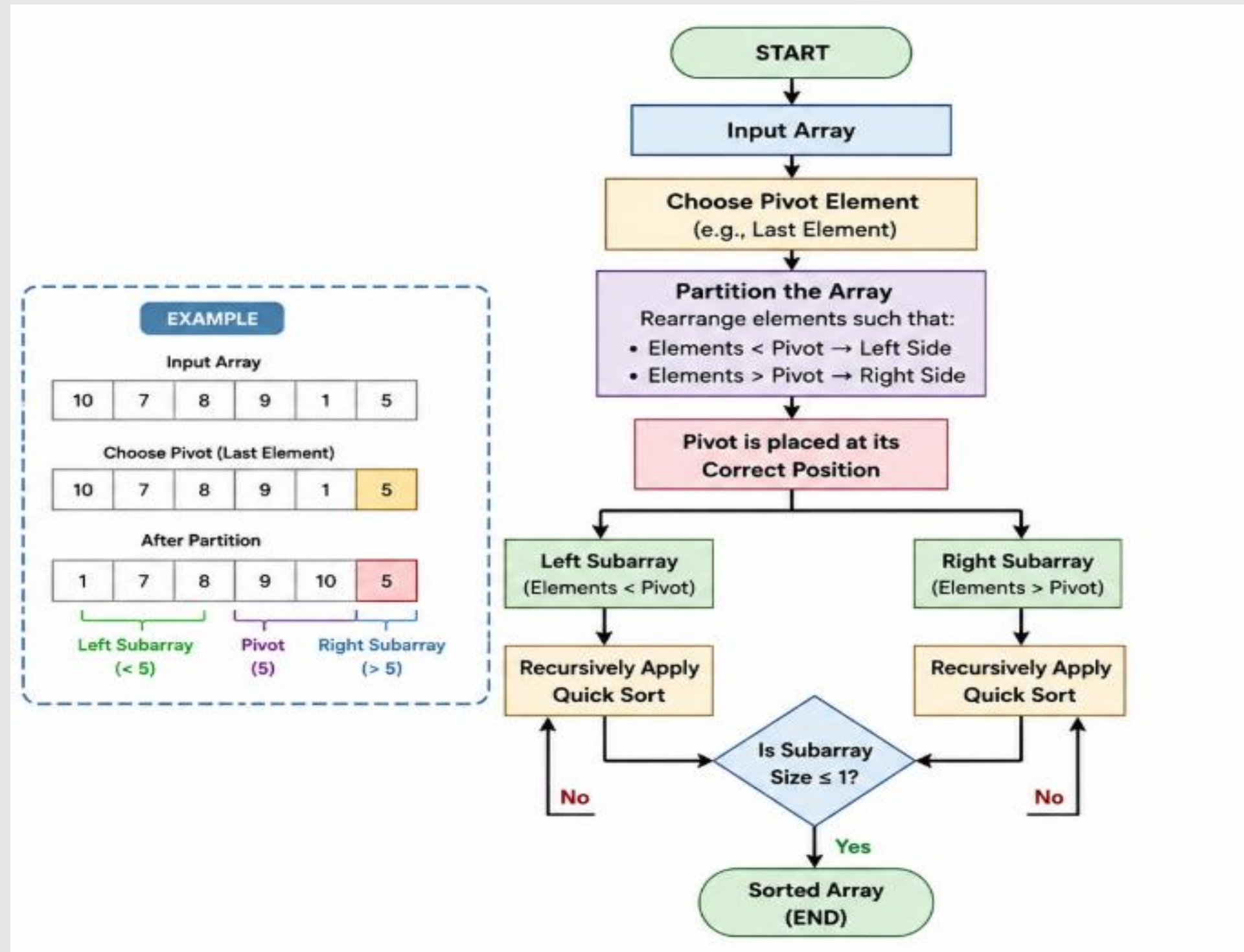
- Analyze the efficiency of the Quick Sort algorithm under Best, Average, and Worst Case scenarios.
- Identify the input conditions that lead to each case.
- Study the effect of different pivot selection strategies on performance.
- Compare the time complexity using asymptotic notation.
- Determine which case has the greatest impact on real-world applications



ALGORITHM

- Algorithm: Quick Sort
- Technique: Divide and Conquer
- Pivot Selection: Last Element
- Sorting Type: In-place Comparison Sort
- Working: Partition → Recursion → Sorted Array
- Average Time Complexity: $O(n \log n)$

SYSTEM ARCHITECTURE



SAMPLE CODE & OUTPUT

```
#include <stdio.h>
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
void quickSort(int arr[], int low, int high)
{
    if (low < high)
```

Enter the number of elements: 6

Enter 6 elements:

71

25

68

413

7118

12

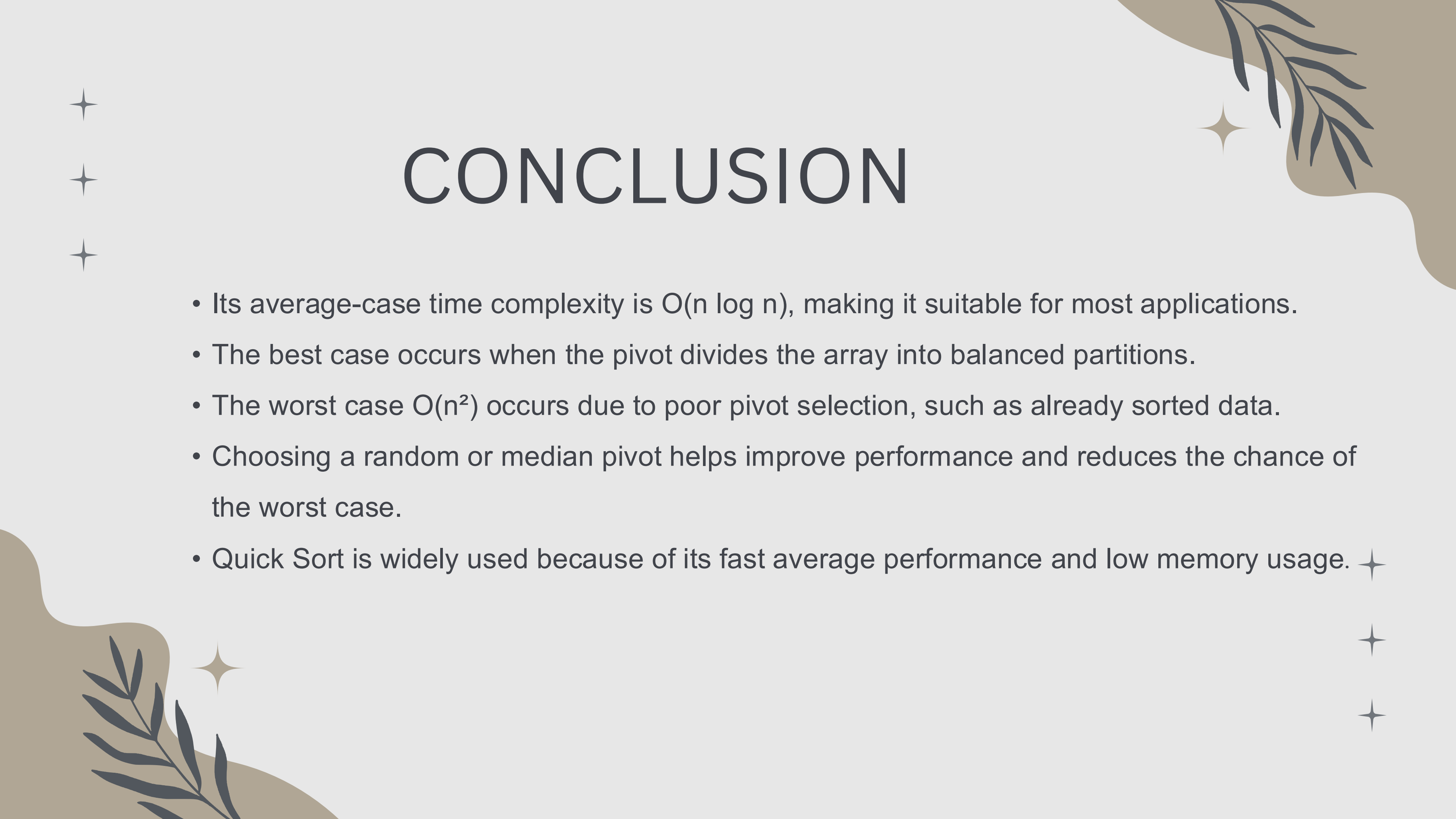
Sorted Array:

12 25 68 71 413 7118

=== Code Execution Successful ===

COMPLEXITY ANALYSIS

Case	Input Condition	Pivot Selection	Time Complexity
Best Case	Pivot divides array equally	Middle/Median Pivot	$O(n \log n)$
Average Case	Random input	Random Pivot	$O(n \log n)$
Worst Case	Already sorted or reverse sorted	First/Last Pivot	$O(n^2)$



CONCLUSION

- Its average-case time complexity is $O(n \log n)$, making it suitable for most applications.
- The best case occurs when the pivot divides the array into balanced partitions.
- The worst case $O(n^2)$ occurs due to poor pivot selection, such as already sorted data.
- Choosing a random or median pivot helps improve performance and reduces the chance of the worst case.
- Quick Sort is widely used because of its fast average performance and low memory usage. ✨



REFERENCES



- THOMAS H. CORMEN, CHARLES E. LEISERSON, RONALD L. RIVEST, AND CLIFFORD STEIN, INTRODUCTION TO ALGORITHMS, 4TH EDITION, MIT PRESS.
 - Ellis Horowitz, Sartaj Sahni, and Susan Anderson-Freed, Fundamentals of Data Structures in C.
 - Mark Allen Weiss, Data Structures and Algorithm Analysis in C.
 - GeeksforGeeks – Quick Sort Tutorial.
 - Programiz – Quick Sort Algorithm.
 - TutorialsPoint – Data Structures and Algorithms.
- 
- 

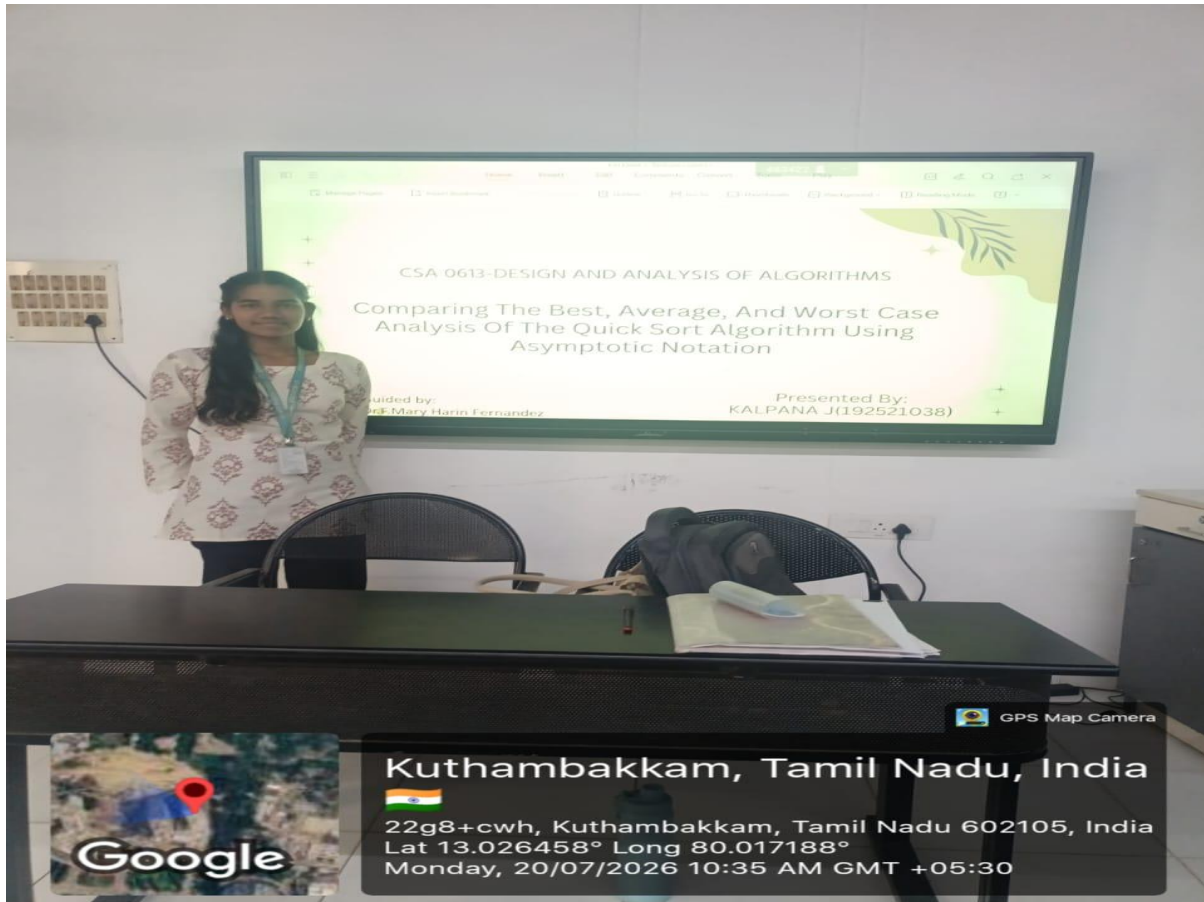
THANK YOU

CO1-AT3

TECHNICAL PRESENTATION

NAME:KALPANA J

REG NO:192521038



Quick Sort is a divide-and-conquer sorting algorithm whose performance varies depending on how the pivot divides the array. In the best case, the pivot splits the array into two equal halves, resulting in a time complexity of $O(n \log n)$. In the average case, the partitions are reasonably balanced, so the algorithm also runs in $O(n \log n)$. In the worst case, the pivot always becomes the smallest or largest element, creating highly unbalanced partitions, and the time complexity increases to $O(n^2)$. Using asymptotic notation (Big-O, Theta, and Omega), these cases help analyze the algorithm's efficiency for different input scenarios, showing that Quick Sort is generally one of the fastest sorting algorithms in practice, despite its worst-case behavior.